
REST API and Web Services

Topics

What is REST API

Designing RESTful APIs

JSON

Other Web Services

gRPC, GraphQL

What's an API?

An **API**, or **Application Programming Interface**, is a set of definitions and protocols that allow one application to communicate with another application

It's sometimes referred to as a **contract between an information provider and an information user**—establishing the content required from the consumer (the call) and the content required by the producer (the response)

What is REST?

In 2000, Roy Fielding proposed **REpresentational State Transfer (REST)** as an architectural approach to designing web services

REST is an architectural style for building distributed systems based on hypermedia

REST is independent of any underlying protocol and is not necessarily tied to HTTP

however, **most common REST API implementations use HTTP** as the application protocol

Main Design Principles of RESTful APIs

REST APIs are designed around **resources**,

which are any kind of **object**, **data**, or **service** that can be **accessed by the client**

A resource has an **identifier**, which is a **URI** that uniquely identifies that resource

For example, the **URI** for a particular customer order might be

`https://example.com/orders/1`

Main Design Principles of RESTful APIs

Clients interact with a service by **exchanging representations** of **resources**

many web APIs use **JSON** as the exchange format

E.g, a **GET** request to the **URI** <https://example.com/orders/1> returns the following **JSON** response body

```
{
  "orderId":1,
  "orderValue":99.90,
  "productId":1,
  "quantity":1
}
```

Main Design Principles of RESTful APIs

REST APIs use a **uniform interface**, which helps to **decouple the client and service implementations**

For REST APIs built on **HTTP**, the uniform interface includes using standard **HTTP** verbs to perform operations on resources

The most common operations are **GET**, **POST**, **PUT**, **PATCH**, and **DELETE**

Main Design Principles of RESTful APIs

REST APIs use a **stateless** request model

HTTP requests should be independent and may occur in any order, so keeping transient state information between requests is not feasible

the only place where **information is stored is in the resources themselves**, and each request should be an atomic operation

this constraint enables web services to be **highly scalable**, because there is no need to retain any affinity between clients and specific servers

any server can handle any request from any client

Main Design Principles of RESTful APIs

REST APIs are driven by **hypermedia links** that are **contained in the representation**

For example, the next slide shows a JSON representation of an order

It contains links to get or update the customer associated with the order

Maturity Model for Web APIs

In 2008, Leonard Richardson proposed the following maturity model for web APIs:

Level 0: Define one URI, and all operations are POST requests to this URI

Level 1: Create separate URIs for individual resources

Level 2: Use HTTP methods to define operations on resources

Level 3: Use hypermedia (HATEOAS)

Maturity Model for Web APIs

Level 3 corresponds to a truly RESTful API according to Fielding's definition

In practice, many published web APIs fall somewhere around level 2

REST API Design Considerations

Organize the API design around resources

Define API operations in terms of HTTP methods

Conform to HTTP Semantics

Support Partial Responses for Large Binary Resources

Use HATEOAS to Enable Navigation to Related Resources

Versioning a RESTful web API

Organize the API design around resources

Focus on the business entities that the web API exposes

for example, in an e-commerce system, the primary entities might be customers and orders

creating an order can be achieved by sending an **HTTP POST** request that contains the order information

the **HTTP response** indicates whether the order was placed successfully or not

when possible, resource **URIs should be based on nouns** (the **resource**) and **not verbs** (the operations on the resource)

Organize the API design around resources

Focus on the business entities that the web API exposes

`https://example.com/orders // Good`

`https://example.com/create-order // Avoid`

Organize the API design around resources

A resource doesn't have to be based on a single physical data item

For example, an order resource might be implemented internally as several tables in a relational database, but presented to the client as a single entity

Avoid creating APIs that simply mirror the internal structure of a database

the purpose of REST is to model entities and the operations that an application can perform on those entities

a client should not be exposed to the internal implementation

Organize the API design around resources

Entities are often grouped together into collections (orders, customers)

A collection is a separate resource from the item within the collection, and should have its own URI

for example, the following URI might represent the collection of orders:

`https://example.com/orders`

Organize the API design around resources

Sending an **HTTP GET** request to a **collection URI** retrieves a list of items in the collection

each item in the collection also has its own unique **URI**

an **HTTP GET** request to the item's **URI** returns the details of that item

Organize the API design around resources

Adopt a consistent naming convention in URIs

in general, it helps to use **plural nouns for URIs that reference collections**

it's a good practice to organize URIs for collections and items into a hierarchy. For example,

/customers is the path to the customers collection, and

/customers/5 is the path to the customer with **ID** equal to **5**

Organize the API design around resources

Consider the relationships between different types of resources and how you might expose these associations

for example, the **`/customers/5/orders`** might represent **all of the orders for customer 5**

you could also go in the other direction, and represent the association from an order back to a customer with a URI such as **`/orders/99/customer`**

Organize the API design around resources

Avoid requiring resource URIs more **complex** than
`collection/item/collection`

for example, avoid doing the following:

`/customers/1/orders/99/products`

Organize the API design around resources

All web requests impose a load on the web server, the more requests, the bigger the load

therefore, try to **avoid "chatty" web APIs** that expose a large number of small resources

Combine related information into bigger resources that can be retrieved with a single request

however, you need to balance this approach against the **overhead of fetching data that the client doesn't need**

Organize the API design around resources

Avoid introducing dependencies between the web API and the underlying data sources

for example, if your data is stored in a relational database, the web API doesn't need to expose each table as a collection of resources

think of the web API as an abstraction of the database

Organize the API design around resources

It might not be possible to map every operation implemented by a web API to a specific resource

You can handle such **non-resource scenarios** through **HTTP requests that invoke a function and return the results as an HTTP response message**

for example, a web API that implements simple calculator operations such as add and subtract could provide URIs that expose these operations as pseudo resources and use the query string to specify the parameters required

Define API operations in terms of HTTP methods

HTTP GET Method retrieves a **representation** of the **resource** at the specified **URI**

the **body** of the **response** message contains the details of the **requested resource**

GET /customers	Retrieve all customers
GET /customers/1	Retrieve the details for customer 1
GET /customers/1/orders	Retrieve all orders for customer 1

Define API operations in terms of HTTP methods

HTTP POST Method creates a new **resource** at the specified **URI**

the **body** of the **request message** provides the details of the new **resource**

POST /customers	Create a new customer
POST /customers/1	ERROR
POST /customers/1/orders	Create a new order for customer 1

Define API operations in terms of HTTP methods

HTTP **POST** Method

note that **POST** can also be used to trigger operations that don't actually create resources

Define API operations in terms of HTTP methods

HTTP PUT Method either **creates** or **replaces** the **resource** at the specified **URI**

the **body** of the **request** message specifies the **resource** to be created or updated

PUT <code>/customers</code>	Bulk update of customers
PUT <code>/customers/1</code>	Update the details of customer 1 if it exists
PUT <code>/customers/1/orders</code>	Bulk update of orders for customer 1

Define API operations in terms of HTTP methods

HTTP PATCH Method performs a **partial update** of a **resource**

the **request body** specifies the set of changes to apply to the **resource**

PATCH /customers	Bulk update of customers
PATCH /customers/1	Update the details of customer 1 if it exists
PATCH /customers/1/orders	Bulk update of orders for customer 1

Define API operations in terms of HTTP methods

HTTP DELETE Method removes the **resource** at the specified **URI**

DELETE /customers	Remove all customers
DELETE /customers/1	Remove customer 1
DELETE /customers/1/orders	Remove all orders for customer 1

The differences between POST, PUT, and PATCH

A **POST** request creates a resource

The **server assigns a URI for the new resource**, and returns that **URI** to the client

In the **REST** model, you frequently apply **POST** requests to collections

the new resource is added to the collection

A **POST** request can also be used to submit data for processing to an existing resource, without any new resource being created

The differences between POST, PUT, and PATCH

A **PUT** request **creates** a **resource** or **updates** an existing resource

The client specifies the **URI** for the **resource**

The request body contains a complete representation of the resource

If a resource with this **URI** already exists, it is replaced

otherwise a new resource is created

PUT requests are most frequently applied to resources that are individual items, such as a specific customer, rather than collections

The differences between POST, PUT, and PATCH

A **PATCH** request performs a **partial update** to an existing **resource**

The client specifies the **URI** for the resource

The request body specifies a set of changes to apply to the resource

This can be more efficient than using **PUT**, because the client only sends the changes, not the entire representation of the resource

The differences between POST, PUT, and PATCH

PUT requests must be **idempotent**

if a client submits the same **PUT** request multiple times, the results should always be the same (the same resource will be modified with the same values)

POST and **PATCH** requests are not guaranteed to be idempotent

<https://www.rfc-editor.org/rfc/rfc6902>

Conform to HTTP Semantics

Media Types

In the HTTP protocol, formats are specified through the use of **media types**, also called **MIME types**

The **Content-Type** header in a request or response specifies the **format of the representation**

Conform to HTTP Semantics

Media Types

```
POST https://example.com/orders HTTP/1.1
Content-Type: application/json; charset=utf-8
Content-Length: 57

{"Id":1, "Name":"Gizmo", "Category":"Widgets", "Price":1.99}
```

If the server doesn't support the media type, it should return HTTP status code 415 (Unsupported Media Type)

Conform to HTTP Semantics

Media Types

A client request can include an **Accept header** that contains a list of media types the client will accept from the server in the response message

```
GET https://example.com/orders/2 HTTP/1.1  
Accept: application/json
```

Conform to HTTP Semantics

Media Types

```
POST https://example.com/orders HTTP/1.1
Content-Type: application/json; charset=utf-8
Content-Length: 57

{"Id":1, "Name":"Gizmo", "Category":"Widgets", "Price":1.99}
```

If the server cannot match any of the media type(s) listed, it should return HTTP status code **406 (Not Acceptable)**

Conform to HTTP Semantics

GET Methods

A **successful GET** method typically returns HTTP status code **200 (OK)**

If the **resource cannot be found**, the method should return **404 (Not Found)**

Should be idempotent, the same **GET** request repeated over the same resource should result in the same state

Conform to HTTP Semantics

POST Methods

If a **POST** method **creates a new resource**, it returns HTTP status code **201 (Created)**

the `URI` of the new resource is included in the **Location** header of the response

the response **body** contains a **representation** of the **resource**

Conform to HTTP Semantics

POST Methods

if the **method does some processing but does not create a new resource**, the method can return HTTP status code **200 (Ok)** and **include the result of the operation in the response body**

Alternatively, if there is no result to return, the method can return HTTP status code **204 (No Content)** with **no response body**

Conform to HTTP Semantics

POST Methods

if the **client puts invalid data into the request**, the server should return HTTP status code **400 (Bad Request)**

the response body can contain additional information about the error or a link to a URI that provides more details

Conform to HTTP Semantics

PUT Methods

if a **PUT** method **creates** a **new resource**, it returns HTTP status code **201 (Created)**, as with a **POST** method

If the method updates an existing resource, it returns either **200 (OK)** or **204 (No Content)**

In some cases, it might **not be possible to update an existing resource**.

in that case, consider returning HTTP status code **409 (Conflict)**

Conform to HTTP Semantics

PUT Methods

Consider implementing bulk HTTP **PUT** operations that can batch updates to multiple resources in a collection

the **PUT** request should specify the **URI** of the collection, and the request body should specify the details of the resources to be modified

this approach can help to reduce chattiness and improve performance

Conform to HTTP Semantics

PATCH Methods

with a **PATCH request**, the client sends a set of updates to an existing **resource**, in the form of a patch document the server processes the patch document to perform the update

the patch document doesn't describe the whole resource, only a set of changes to apply

Conform to HTTP Semantics

PATCH Methods

there are two main **JSON-based patch formats**, called

JSON Patch and **JSON Merge Patch**

Conform to HTTP Semantics

JSON MERGE PATCH

The patch document has the same structure as the original JSON resource, but includes just the subset of fields that should be changed or added

```
{
  "name": "gizmo",
  "category": "widgets",
  "color": "blue",
  "price": 10
}
```

Original

```
{
  "price": 12,
  "color": null,
  "size": "small"
}
```

Patch Document

Conform to HTTP Semantics

JSON MERGE PATCH

A **field can be deleted by specifying `null`** for the field value in the patch document. (That means merge patch is not suitable if the original resource can have explicit null values)

```
{  
  "name": "gizmo",  
  "category": "widgets",  
  "color": "blue",  
  "price": 10  
}
```

Original

```
{  
  "price": 12,  
  "color": null,  
  "size": "small"  
}
```

Patch Document

Conform to HTTP Semantics

JSON MERGE PATCH

The **media type** for JSON merge patch is

`application/merge-patch+json`

Conform to HTTP Semantics

JSON PATCH

It specifies the changes as a sequence of **operations** to apply

Operations include **add**, **remove**, **replace**, **copy**, and **test** (to validate values)

The **media type** for JSON patch is **application/json-patch+json**

Conform to HTTP Semantics

PATCH Methods

Error conditions and Corresponding HTTP Status Code for **PATCH** requests

Error Conditions	HTTP Status Code
The patch document format isn't supported	415 (Unsupported Media Type)
Malformed patch document	400 (Bad Request)
The patch document is valid, but the changes can't be applied to the resource in its current state	409 (Conflict)

Conform to HTTP Semantics

DELETE Methods

If the **delete operation** is **successful**, the web server should respond with HTTP status code **204 (No Content)**, indicating that the process has been successfully handled, but that the response body contains no further information

If the **resource doesn't exist**, the web server can return HTTP **404 (Not Found)**

Conform to HTTP Semantics

DELETE Methods

should be idempotent

sending multiple **DELETE** requests to the same URI should have the same effect, although the HTTP status code in the response messages may be different

the first **DELETE** request might return status code **204 (No Content)**, while a subsequent **DELETE** request might return status code **404 (Not Found)**

Conform to HTTP Semantics

Asynchronous Operations

Sometimes a **POST**, **PUT**, **PATCH**, or **DELETE** operation might require processing that takes a while to complete

return HTTP status code **202** (**Accepted**) to indicate the request was accepted for processing but is not completed

Conform to HTTP Semantics

Asynchronous Operations

You should expose an **endpoint that returns the status of an asynchronous request**, so the client can monitor the status by polling the status endpoint

Include the **URI** of the **status endpoint** in the **Location** header of the **202** response. For example:

```
HTTP/1.1 202 Accepted
Location: /api/status/12345
```

Conform to HTTP Semantics

Asynchronous Operations

If the client sends a **GET request to the status endpoint**, the **response** should contain **the current status of the request**

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "In progress",
  "link": { "rel": "cancel", "method": "delete", "href": "/api/status/12345" }
}
```

Conform to HTTP Semantics

Asynchronous Operations

If the asynchronous operation **creates a new resource**, the status endpoint should return status code **303** (**See Other**) after the operation completes

In the **303** response, include a **Location** header that gives the **URI of the new resource**

```
HTTP/1.1 303 See Other
Location: /api/orders/12345
```

Conform to HTTP Semantics

Filter, Paginate, and Sort Data

API should allow, when necessary, passing a filter in the query string of the URI

In the following example the web API is responsible for parsing and handling the **minCost** parameter in the query string and **returning the filtered results** on the server side

```
/orders?minCost=n
```

Conform to HTTP Semantics

Filter, Paginate, and Sort Data

GET requests over collection resources can potentially return a large number of items

You should design a web API to **limit** the amount of data returned by any single request

```
/orders?limit=25&offset=50
```

Conform to HTTP Semantics

Filter, Paginate, and Sort Data

To assist client applications, **GET** requests that **return paginated data should also include** some form of metadata that indicate the **total number of resources available** in the collection

You can use a similar strategy to sort data as it is fetched, by providing a **sort** parameter that takes a field name as the value, such as `/orders?sort=ProductID`

Conform to HTTP Semantics

Filter, Paginate, and Sort Data

You can extend this approach to limit the fields returned for each item, if each item contains a large amount of data

for example, you could use a query string parameter that accepts a comma-delimited list of fields, such as

```
/orders?fields=ProductID,Quantity
```

Conform to HTTP Semantics

Filter, Paginate, and Sort Data

Give all optional parameters in query strings meaningful defaults. For example

set the limit parameter to 10 and the **offset parameter to 0** if you implement pagination,

set the **sort parameter to the key of the resource** if you implement ordering, and

set the **fields parameter to all fields in the resource** if you support projections

Support Partial Responses for Large Binary Resources

A resource may contain large binary fields, such as files or images

To overcome problems caused by unreliable and intermittent connections and to improve response times, consider enabling such resources to be retrieved in chunks

To do this, the web API should support the **Accept-Ranges** header for **GET** requests for large resources

Support Partial Responses for Large Binary Resources

The **Accept-Ranges** header indicates that the **GET** operation supports partial requests

The client application can submit **GET** requests that return a subset of a resource, specified as a range of bytes

Support Partial Responses for Large Binary Resources

Also, consider implementing HTTP **HEAD** requests for these resources

A **HEAD** request is similar to a **GET** request, except that it only returns the HTTP headers that describe the resource, with an empty message body

A client application can issue a **HEAD request to determine whether to fetch a resource by using partial **GET** requests**

Support Partial Responses for Large Binary Resources

Example **HEAD** request

```
HEAD https://example.com/products/10?fields=prodImage HTTP/1.1
```

Here is an example response message

```
HTTP/1.1 200 OK
```

```
Accept-Ranges: bytes
```

```
Content-Type: image/jpeg
```

```
Content-Length: 4580
```

The **Accept-Ranges** header indicates that the corresponding GET operation supports partial results

The **Content-Length** header gives the total size of the resource

Support Partial Responses for Large Binary Resources

The client application can use the response of the HEAD request to retrieve the data in smaller chunks

```
GET https://example.com/products/10?fields=prodImage HTTP/1.1
Range: bytes=0-2499
```

The response message

```
HTTP/1.1 206 Partial Content
```

```
Accept-Ranges: bytes
```

```
Content-Type: image/jpeg
```

```
Content-Length: 2500
```

```
Content-Range: bytes 0-2499/4580
```

```
[...]
```

Use HATEOAS to Enable Navigation to Related Resources

Each HTTP **GET** request should return the information necessary to find the resources related directly to the requested object through **hyperlinks included in the response**, and it should also be provided with information that describes the **operations available on each of these resources**

This principle is known as **HATEOAS**, or **H**ypertext **a**s **t**he **E**ngine **o**f **A**pplication **S**tate

An example is shown in the next slide

```
{
  "orderId":3,
  "productId":2,
  "quantity":4,
  "orderValue":16.60,
  "links":[
    {
      "rel":"customer",
      "href":"https://example.com/customers/3" ,
      "action":"PUT",
    }
  ],
  "types":["application/x-www-form-urlencoded" ]
},
{
  "rel":"self",
  "href":"https://example.com/orders/3" ,
  "action":"DELETE",
  "types":[]
}]
}
```

Versioning a RESTful web API

Versioning enables a web API to indicate the features and resources that it exposes, and a client application can submit requests that are directed to a specific version of a feature or resource

Question

What will happen to clients who consume an API such as the following, if you change the schema of the resource

`https://example.com/api/customers`

Versioning a RESTful web API

URI Versioning

Each time you modify the web API or change the schema of resources, you add a version number to the URI for each resource

```
https://example.com/api/v2/customers/3
```

Versioning a RESTful web API

Query String Versioning

You can specify the version of the resource by using a parameter within the query string appended to the HTTP request

```
https://example.com/customers/3?version=2
```

Versioning a RESTful web API

Header Versioning

You can implement a **custom header** that indicates the version of the resource

```
GET https://example.com/customers/3 HTTP/1.1  
Custom-Header: api-version=1
```

Open API Initiative

The **Open API Initiative** was created by an industry consortium **to standardize REST API descriptions** across vendors

As part of this initiative, the **Swagger 2.0 specification** was renamed the **OpenAPI Specification (OAS)** and brought under the Open API Initiative

OpenAPI promotes a **contract-first approach**, rather than an implementation-first approach

Tools like Swagger can generate client libraries or documentation from API contracts

REST APIs in NestJs

NestJs allows you to create a **RESTful APIs**

Sample Code

CRUD operation in NestJs

Question

How do you consume **REST APIs**?

JSON (ECMA-404, RFC-8259)

JavaScript Object Notation (JSON) is a **lightweight, text-based, language independent data interchange format**

It is a text format for the serialization of structured data

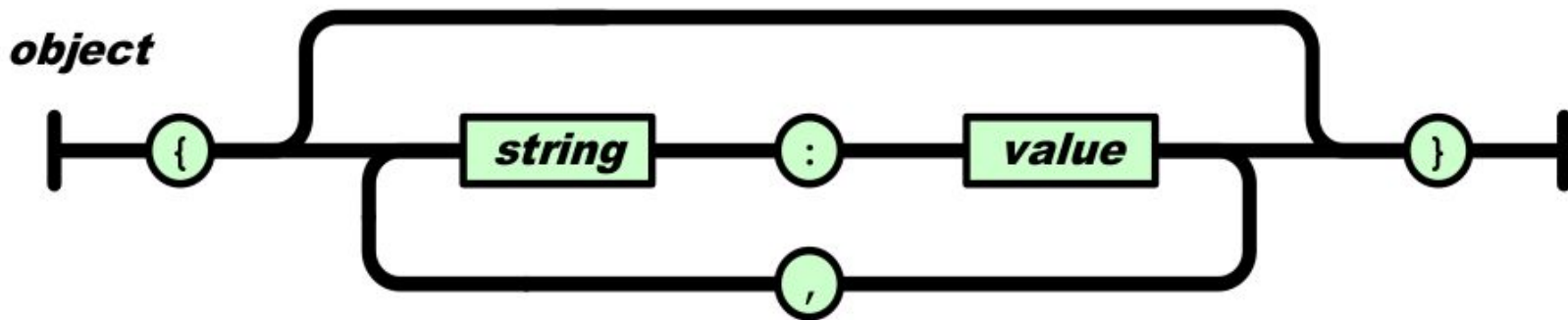
JSON can represent **four primitive types** (**strings**, **numbers**, **booleans**, and **null**) and **two structured types** (**objects** and **arrays**)

A **string** is a sequence of zero or more Unicode characters wrapped with quotation marks

JSON (ECMA-404, RFC-8259)

An **object** structure is represented as a **pair of curly brackets** surrounding unordered collection of **zero or more name/value pairs (or members)**

A **name** is a **string** and a **value** is a **string**, **number**, **boolean**, **null**, **object**, or **array**

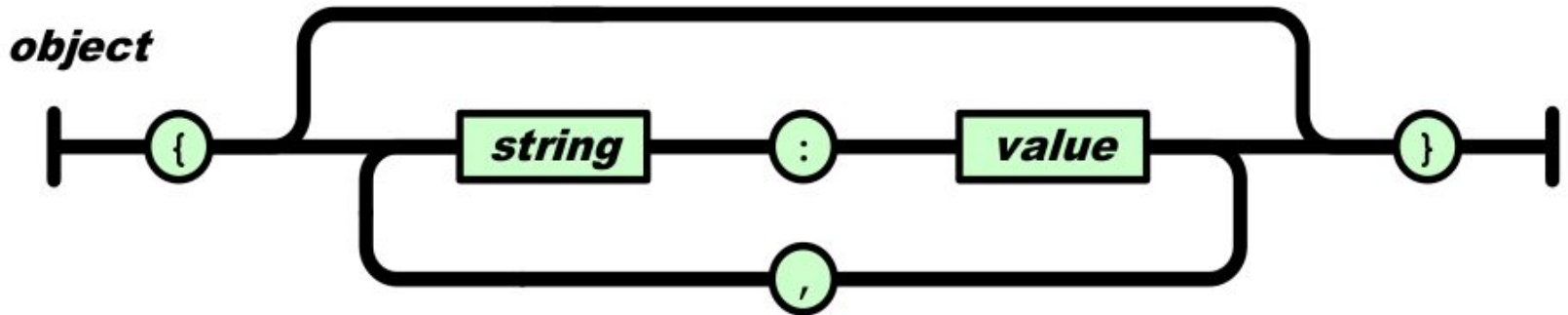


JSON (ECMA-404, RFC-8259)

Which of the following is a JSON according to the diagram

`{ }`

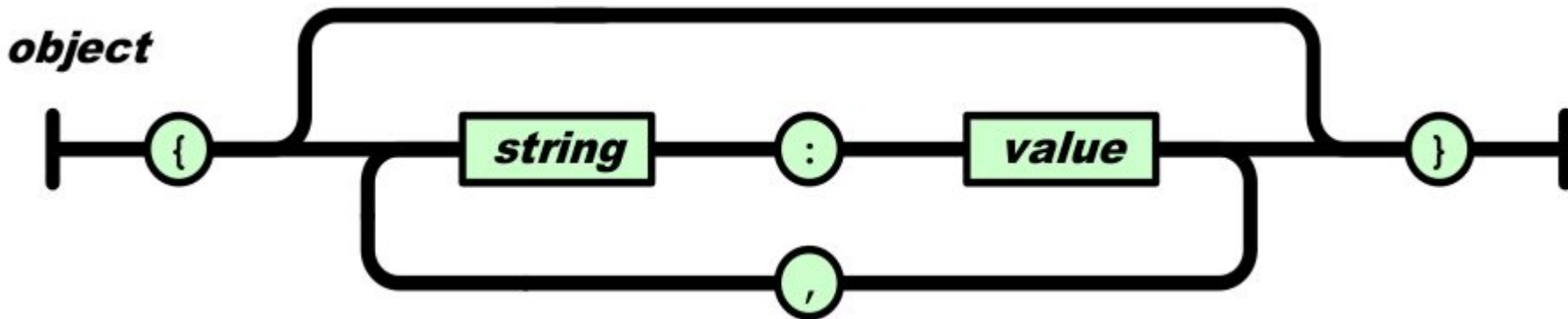
`{ "": { } }`



JSON (ECMA-404, RFC-8259)

A **single colon** comes **after each name**, separating the **name** from the **value** and a **single comma** separates a **value** from a **following name**

The **names** within an **object** **SHOULD** be unique



JSON (ECMA-404, RFC-8259)

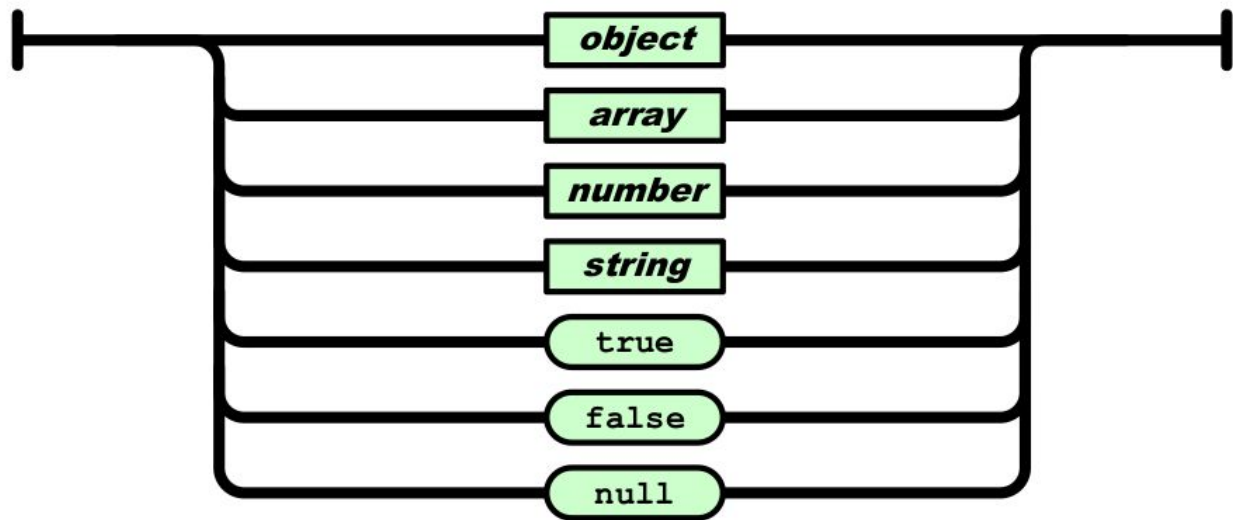
Is the following correct JSON?

```
{  
  "": {  
    "1": "one",  
    "2": "two",  
    "1": "ONE"  
  }  
}
```

JSON (ECMA-404, RFC-8259)

A **value** MUST be an **object**, **array**, **number**, or **string**, or one of the following **three literal names**: **false**, **null**, **true**

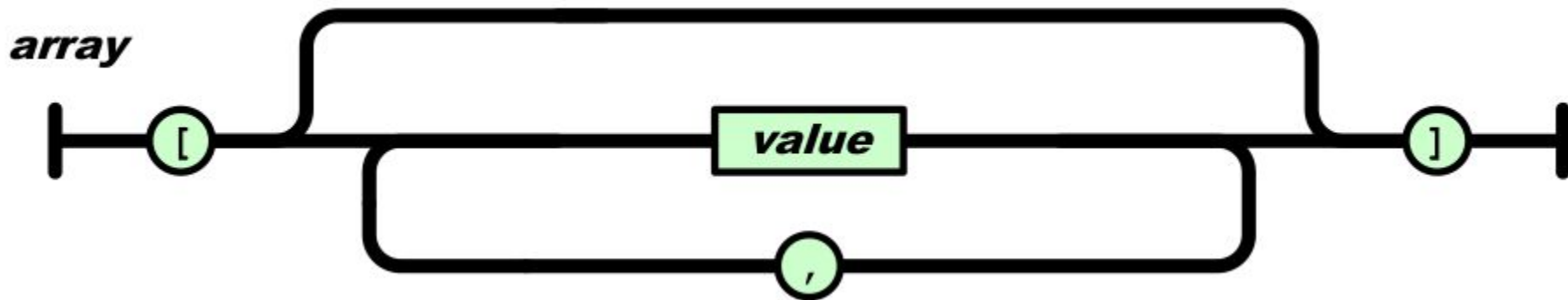
value



The literal names
MUST be lowercase

JSON (ECMA-404, RFC-8259)

An **array** is an ordered sequence of zero or more **values**

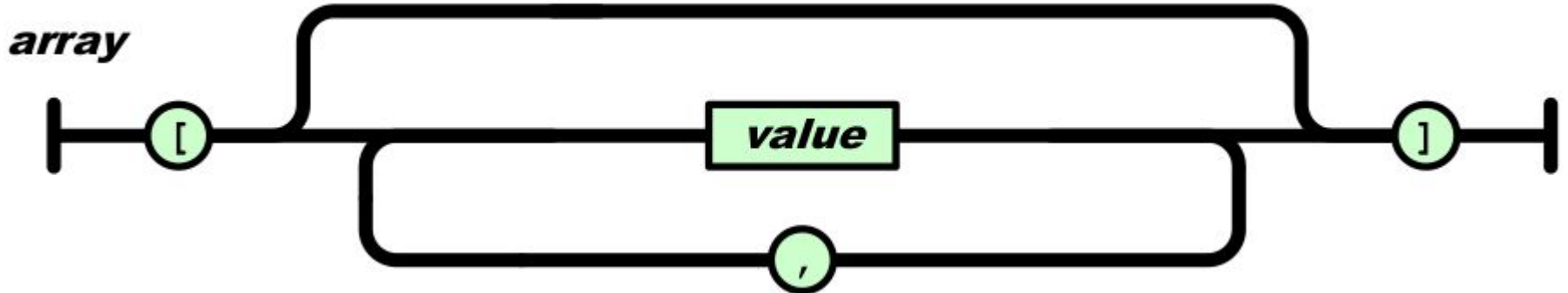


JSON (ECMA-404, RFC-8259)

Which one of the following are correct JSON according to the definition shown below?

[]

[{ }]



JSON Object Example

```
{  
  "Image": {  
    "Width": 800,  
    "Height": 600,  
    "Title": "View from 15th Floor",  
    "Thumbnail": {  
      "Url": "http://www.expl.com/img/4",  
      "Height": 125,  
      "Width": 100  
    },  
    "Animated": false,  
    "IDs": [116, 943, 234, 38793]  
  }  
}
```

Example JSON array containing two objects

```
[  
  {  
    "precision": "zip",  
    "Latitude": 37.7668,  
    "Longitude": -122.3959,  
    "Country": "US"  
  },  
  {  
    "precision": "zip",  
    "Latitude": 37.371991,  
    "Longitude": -122.026020,  
    "Country": "US"  
  }  
]
```

Question

Which of the following are JSONs

`true`

`False`

`""`

`hello`

`"Hello"`

`null`

Other Web Services

Remote Procedure Call (RPC) is one of the technology that allow one program to interact with another (SOAP is RPC like)

The **RPC** model tries to **make a request to a remote network service look the same as calling a function or method** in your programming language, **within the same process** (this abstraction is called **location transparency**)

Other Web Services

Some of the example of recent Web Service related technologies includes

gRPC from Google

GraphQL from Facebook



gRPC

In **gRPC** a client application can directly call methods on a server application on a different machine as if it was a local object

As in many **RPC** systems, **gRPC** is based around the idea of **defining a service (an interface), specifying the methods that can be called remotely** with their parameters and return types

On the server side, the server implements this interface and runs a **gRPC** server to handle client calls

On the client side, the client has a stub that provides the same methods as the server

gRPC Example

```
// The greeting service definition.
```

```
service Greeter {
```

```
    // Sends a greeting
```

```
    rpc SayHello (HelloRequest) returns (HelloReply) {}
```

```
}
```

gRPC Example

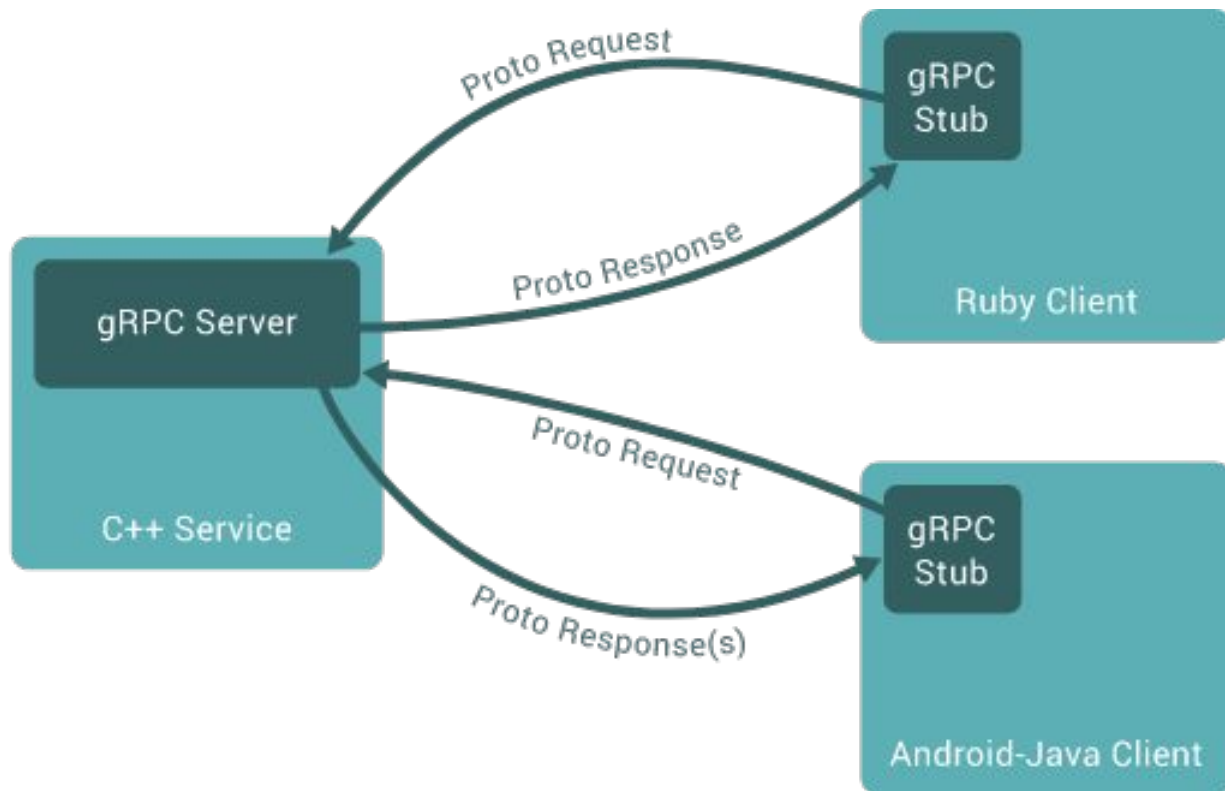
```
// The request message containing the user's name.
```

```
message HelloRequest {  
    string name = 1;  
}
```

```
// The response message containing the greetings
```

```
message HelloReply {  
    string message = 1;  
}
```

gRPC



gRPC and Protocol Buffers

Protocol Buffer is Google's open source mechanism for **serializing** structured data

By default, **gRPC** uses **protocol buffers as the Interface Definition Language (IDL)** for describing both the service interface and the structure of the payload messages

gRPC can also be used with other data formats such as **JSON**

Working with Protocol Buffers

The first step when working with protocol buffers is to **define** the **structure** for the data you want to serialize in a **proto** file: this is an ordinary text file with a **.proto** extension

```
message Person {  
    string name = 1;  
    int32 id = 2;  
    bool has_ponycopter = 3;  
}
```

Working with Protocol Buffers

Once you've specified your data structures, you use the protocol buffer compiler **protoc** to generate data access classes in your preferred language(s) from your proto definition

These provide **simple accessors** for each field, like **name()** and **set_name()**, as well as **methods** to **serialize** the whole structure **to raw bytes** or **parse from raw bytes**

Working with Protocol Buffers

You can then use the generated class in your application to **populate**, **serialize**, and **retrieve** Person protocol buffer messages.

gRPC and Protocol Buffer References

<https://grpc.io/docs/guides/>

<https://developers.google.com/protocol-buffers/docs/gotutorial>

<https://grpc.io/docs/what-is-grpc/core-concepts/>

GraphQL

GraphQL is a **query language for your API**, and a server-side runtime for executing queries by using a type system you define for your data

GraphQL isn't tied to any specific database or storage engine and is instead backed by your existing code and data

GraphQL

A **GraphQL** service is created by defining types, and fields on those types, then providing functions for each field on each type

Once a **GraphQL** service is running (typically at a URL on a web service), it can be sent **GraphQL** queries to validate and execute

A received query is first checked to ensure it only refers to the types and fields defined, then runs the provided functions to produce a result.

GraphQL: Describe your data

```
type Project {  
  name: String  
  tagline: String  
  contributors: [User]  
}
```

GraphQL: Ask for what you want

```
{  
  project(name: "GraphQL") {  
    tagline  
  }  
}
```

GraphQL: Get predictable results

```
{  
  "project": {  
    "tagline": "A query language for APIs"  
  }  
}
```

GraphQL References

`https://graphql.org/`

`https://graphql.org/learn/`